The Tower of Babel



Tales of a Traveller – Part 4

The Mathematics of Large Language Models

By Jai Sharma

Journey through the woods of Artificial Intelligence & Machine Learning

Legend has it that the Babylonians wanted to build a tower so mighty, that it reached the heavens. They could then be like God and would not need Him anymore. However, God disrupted their plans by so confusing the language of the workers, that they no longer could understand one another.

Over the years an entire genre of literature has been devoted to communication. Diplomatic communication, Legal communication, Organizational communication, Comms of all sorts and in all settings. Add to this, the plethora of languages and dialects, accents and cultural nuances – this poses a formidable challenge in the domain of *Natural Language Processing* (NLP).

With apps like ChatGPT and Grok or Claude and Gemini, Large Language Models (LLM) have gained huge popularity. Most organizations are re-thinking how they can leverage GenAI in their day-to-day workflows. In corporate settings, this has expanded beyond simple chatbots to AI agents and vibe coding. Our personal lives too have been revolutionized by Gen AI - from document summarization and voice-controlled smart devices in our homes, to the way we engage in creative pursuits.

1. Communication Challenges faced by LLMs

Whilst experts have devised programs to equip managers improve their communication skills, the focus of this blog is on AI architectures built for machines to understand human communication.

Some of the challenges in this domain are:

- **Polysemy**: A word could have multiple meanings. Take for example, the word 'bank'. It has different meanings based on context.
    - 'I am going to the bank to deposit some money'.
    - 'You can bank on me to complete this task'.
    - 'I am taking a stroll along the South bank of the river'.

  Similarly, the word 'Apple' may refer to a fruit or a technology company or a branded device such as a laptop or a mobile phone!

  Synonyms: Different words having the same meaning

    - Canine and Dogs
    - Feline and Cats
    - Fast and Quick

- **Historical semantics:** There are many examples where meanings of words and expressions change over time. Languages evolve, reflecting the shift in cultural orientation and beliefs. For example, the word 'gay' which meant happy, is used in an entirely different context today.

- **Connotations or intent:** Figures of speech are often times not to be taken literally. For example, "John is too big for his boots" has nothing to do with John's shoe size! Different languages have their own phraseology. How is a machine to know? How can a chatbot sense the users' satire, frustration and intent?

2. Representing Words as Numbers

Whilst words could be mis-interpreted, numerals on the other hand, have a certain precision and clarity. Mathematics and Statistics and by extension, AI models, only deal with numbers – specifically, a matrix of numbers. For instance, computer vision algorithms are trained on a matrix of pixel values. Voice recognition systems use numeric representation of sound through waveforms or spectrograms – amplitude, frequency, wavelength and so on.

But, whilst images and sound have a natural numeric representation, how do we deal with words?

Imagine the words King, Queen, Man and Woman represented as some sort of numeric vectors. Then in a mathematical sense, using Vector algebra one would expect:

- (Vector Queen) would equal (Vector King) minus (Vector Man) plus (Vector Woman)! (*Remove Man from a King and add a Woman, you have a Queen).*

- The vectors 'King' and 'Queen' would be equidistant to the vectors 'Man' and 'Woman'!

Given the issue of polysemy we discussed earlier, any vector representation will need to cater for a word having multiple meanings, based on context.

The current plethora of Large Language Models have found a way to represent words as numbers: ***vectors in a multi-dimensional semantic space*.**

What is this "Semantic space" and how do we get to an appropriate vector representation? How do we build a framework for semantics?

3. Corpus, Tokens and Vocabulary

A "semantic space" may be understood as a multi-dimensional abstract representation where the distance between points represents meaning (semantics) rather than physical space.

The smallest structural unit mapped into this abstract space is called a "**token**". Consider the words: Eat, Play, Carry, Eating, Playing, Carrying. These are six words but will be tokenized as: eat, play, carry, and a suffix 'ing'.

The suffix "ing" when added to the base token '*eat*', creates the token *'eating'* and so on. Similarly, a suffix like 'th' is a token. When it gets added to the words: *four, ten and six* it forms new tokens, fourth, tenth, sixth and so on.

Each punctuation mark is also a distinct token.

Authoritative sources like Wikipedia, technical journals and the like, are inputs to the tokenizing process. These form the *Corpus*. The corpus is tokenized by breaking it down into frequently occurring words and sub-words. The set of unique tokens constitutes the ***Vocabulary***.

To start with all raw alphabets and punctuation marks are included as a token. This is a 1-gram. Next, 2-grams. Two frequently occurring adjacent alphabets like "at", "be", "th", "in", "on" etc. Then 3-grams, which produce suffix combinations like "ing".

The n-gram builder does not slice words arbitrarily. Modern LLM tokenizers implement pre-tokenization. The algorithm identifies common word structures before splitting complex words. Thus while 'th' is identified as a suffix in fourth, tenth, sixth and ninth, the word 'width' will **not** be split as the sequence since 'width' occurs very frequently on its own. So too 'nin' occurs frequently as contiguous characters in words like **nin**e, **nin**eteen, **nin**ety, tan**nin** and so on.

This highlights that tokens are merely **statistical units** and like the token 'nin' have **no** semantic meaning from a human perspective. They are simply, frequently occurring words or sub-words! And because they occur frequently within the corpus, they get added to the vocabulary. This process of n-grams continues until the longest word, or the vocabulary limit has been reached.

Each of the hugely popular LLMs has its own distinctive tokenizer engine and a pre-defined vocabulary size measured in number of tokens.

The GPT family (OpenAI) uses an engine called *tiktoken*. Googles Gemini family uses its own proprietary engine for multilingual capabilities. It also features a large vocabulary of 256,000 tokens to provide for **multilingual and mathematical symbols in a single vocabulary.**

As a token gets added to the vocabulary, it is allotted a running sequential number or a database key called **initial id.**

This initial id is then converted into a vector representation as follows:

Consider a small vocabulary of 50,000 tokens. The token with an initial id 3 is represented as an array of 50,000 values with a 1 in index position 3 and all other positions filled with zeroes. Thus, [0,0,0,1,0,0,0……0]. (Note that the first position is index 0. So, index position 3, is actually the 4th position!)

Such a representation called **binary 1-hot encoding** has some severe limitations:

a) **It is a very sparse vector** (a very long vector with just one position filled and many thousands of zeroes)
b) **It has no semantic representation**. The initial id is allocated as a sequential number. This does not ensure that the token 'pup' and the token 'dog' or 'canines' will be close together, for example.
c) **It is static** and cannot cater for polysemy (multiple meanings). In fact, as it stands it has no particular meaning.

Thus, whilst we have converted tokens to vectors with large number of dimensions, these dimensions have absolutely no semantic meaning.

Let us look at this initial mathematical representation of tokens as 1-hot encoded vectors. Our example vocabulary of 50,000 tokens can now be represented as a stacked matrix of 50,000 X 50,000 1-hot encoded vectors!

Token 0:	[1,0,0,0	0]
Token 1:	[0,1,0,0,0	0]
Token 2:	[0,0,1,0,0,0	0]
Token 3:	[0,0,0,1,0,0,0	0]
Token 5:	[0,0,0,0,1,0,0,0	0]
Token 49,999	[0,0,0,0,0,0,0,0	1]

Note how our vocabulary comprising of 50,000 tokens arranged sequentially, forms an ***identity-matrix say V, of stacked vectors*** (A square matrix with all elements along the main diagonal set to1 and all other elements are 0!).

You will also note that the above vectors are ***orthogonal*** *to each* other (each of them is at a 90-degree orientation to the other). **The dot product of any combination of these vectors is zero.** This suggests there is **no semantic structure** between the various tokens!

Although our vocabulary now has a rather neat mathematical representation, it lacks any semantics. So, at this stage, there can be no meaningful conversation using this matrix.

The challenge is to convert this massive identity matrix into a dense matrix where each vector that corresponds to a token, has some meaning in an abstract semantic space.

Mathematically, vectors have an ***orientation*** and ***magnitude***. We would thus need to rotate these vectors from their current 90-degree isolated orientation into meaningful clusters where *"apples"* and *"bananas"* and various other fruits have similar orientation in a newly defined semantic space. The values or vector length would represent strength or frequency of those concepts.

4. Defining a Semantic Space

What Am I is a popular guessing game that children play. In this game, you read a clue and players ask a maximum of say 20 random questions in an effort to guess what you had in mind. After a while, players formulate their strategy and learn to ask a set of questions in a particular sequence that will help them quickly identify the object.

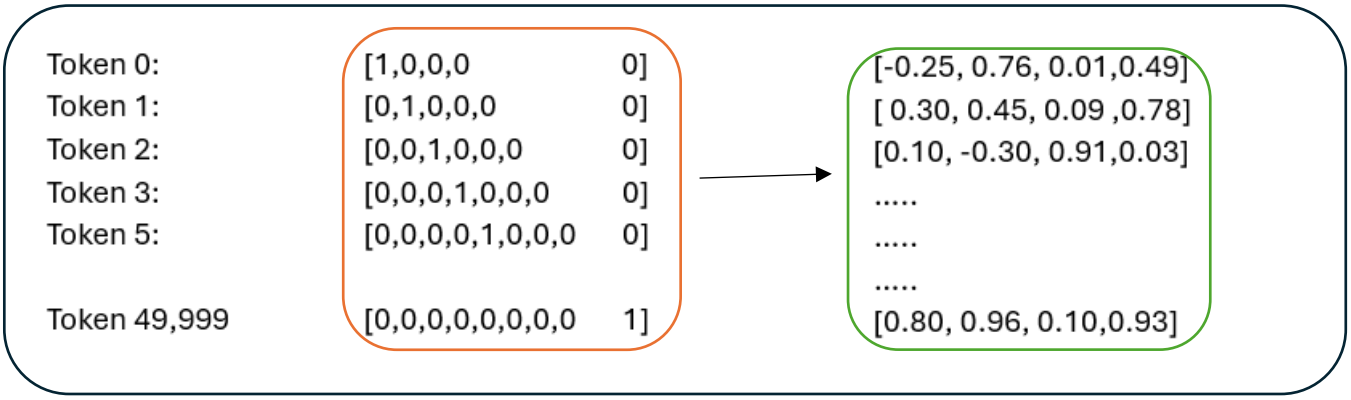
Think of these questions as dimensions in a semantic space. The aim is to define enough dimensions to position words, or tokens, within an evolving abstract space. As the model learns from data in this space, **these dimensions help organise tokens according to patterns of meaning and usage.** As an example, we can easily picture various sized boxes as a vector having three dimensions: *Length, Width, Height*.

We are now wanting to establish some sort of a multi-dimensional framework where we can neatly slot various tokens. Each token would have different values or weights across these dimensions thereby giving it some semantics, some semblance of meaning.

This so called '*semantics'* is a representation of tokens in an abstract mathematical space – **not to be confused with human meaning!**

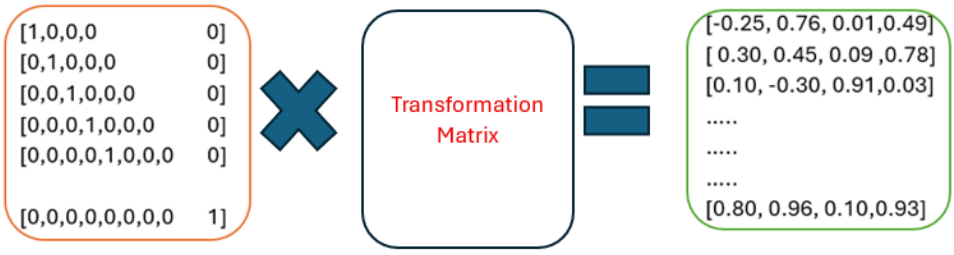
It is quite natural to ask, *how many dimensions and what do these dimensions represent?* Thinking purely abstractly, it hardly matters what these dimensions mean in a 'human sense'. They are **not** handcrafted but are simply a mathematical representation of different features that help place a token. In such an abstract space, we would want *dogs, puppies, canines* to be close together and further apart from *aeroplane, ships, mountains etc.* Similarly, the token *dog* or *puppy* or *pup* would somehow be related to *playful*, *walking*, *loyal*, *barking* etc. In terms of number of dimensions, we would need enough dimensions to represent all possible worldly objects and concepts – think in terms of thousands of dimensions!

In other words, our original matrix of 1-hot encoded vectors that have no meaning are to be transformed into a different vector space with dimensions that give all of those tokens some meaning.



In linear algebra, the operation of **matrix multiplication** is used to transform a matrix from one dimension to another. This requires the following:

- You need to two matrices – the input matrix to be transformed and an operator matrix also called a transformation matrix.



[***Input matrix***] multiplied by [*Transformation matrix*] equals [*Output matrix*]

- The number of rows of the Transformation matrix must equal number of columns of the **Input matrix**

The Output matrix will then have the same number of columns as the Transformation matrix and the same number of **rows** as the **input matrix.**

In this case, the input matrix of 1-hot encoded vectors is a square matrix of shape (v * v) where the columns and rows equate to the vocabulary size.

Some of the modern flagship LLM's like Google Gemini has a vocabulary size of **256,000** tokens and **8,192** dimensions for each token!

The input matrix in such models comprises of 1-hot encoded vectors of shape 256,000 X 256,000.

We are wanting to transform this into a semantic space of 8,192 dimensions. i.e. an output matrix O of shape (256,000 tokens X 8192 dimensions).

To achieve this transformation, we would then need a transformation matrix of shape 256,000 * 8,192

$I_{(256,000 \times 256,000)} \times T_{(256,000 \times 8,192)} = O_{(256,000 \times 8,192)}$

It is worth noting, that since **I** is an ***identity* matrix,** the output matrix **O** is in fact exactly identical to the Transformation matrix **T.**

How do we construct this matrix?

To start with, the elements of the Transformation Matrix $\mathbf{T}_{(256,000 \times 8,192)}$ are initialised with random unconstrained weights (using He/Xavier initialisation). Since the matrix O is identical to the matrix T, we have replaced one set of meaningless 1-hot encoded vectors with another set of equally chaotic random vectors in a different dimension. So how exactly is this going to help?

5. Transformer Architecture: Queries(Q), Keys (K), Value (V)

Circa 2017, a group of eight scientists and engineers working at Google authored a research paper "***Attention is all you need***".

The paper proposed a new architecture known as the ***Transformer***.

For any conversation, you need more than just a vocabulary, you also need 'grammar'.

For instance, in the English language, a verb would need a noun, a noun may need an adjective or other nouns, an adverb needs a verb and so on. One word leads to another via these linkages.

Besides representing vocabulary as a matrix, we would also need to mathematically define this 'grammar' - how various words or tokens are related to each other. Tokens that represent nouns seek other tokens that represent verbs or adjectives or pronouns etc.

Since every language has its own grammatical construct, we are ideally wanting a language agnostic mathematical framework for representing grammar – semantic linkages between words.

Basically, in a text stream, words simultaneously seek context and provide context! Mathematically, these distinct roles need to be represented as two separate matrices in a high-dimensional vector space.

What each token seeks across each dimension could be represented as a vector **q(Query).** What each token provides across each dimension could also be represented as another vector **k(Key).**

Then, vectors **q** form the query matrix **Q.** Vectors **k** form a key matrix **K.**

To further clarify these two new matrices **Q** and **K**, think of a bag containing many tokens representing Jobs and many tokens representing candidates.

Now each Job token is looking for some specifics. Some jobs demand a PhD for instance or, experience of at least 5 years, other jobs are entry-level etc. These requirements may be thought of as vectors **q.**

Each candidate token on the other hand, offers something – Education, Experience, specialised certifications etc. These may be thought of as the vectors **k**.

But candidates also have their own needs: Remote working, salary expectations etc.

So, whilst each token offers something, it also seeks something! i.e. every token has vector **q** and a vector **k!**

This brings us to the vector **v (Value).** This Value vector could be thought of as the token payload. Thus, for a token representing a job, whilst the vector **q** represents what the job demands, the value vector **v** represents the job description!

For human conversation to be intelligible, one needs context. i.e. a certain number of words need to be spoken before the conversation starts making some sense.

Let us assume that the context window for our LLM has an input stream of 32,768 tokens. Since each token has 8192 dimensions, this input stream could be represented as a matrix **X** of dimension [32768 X 8192]. As discussed above, each of these tokens has a query vector and a key vector.

Thinking purely abstractly, consider a matrix of weights W_Q of dimension [8192 X 8192]. Let's call this as the ***Query Projection Matrix*.** The purpose of the query projection matrix is to transform the raw input features **X** to represent what context each token in the input stream seeks. It does this by using the dot product operation to create a new matrix: $\mathbf{Q} = X.\ W_Q$.

W_Q [1,1] would represent the modifier or contribution weight to transform feature 1 of every token to feature 1 in this new space. W_Q [1,2] would represent contribution of feature 1 of every token to feature 2 in this new space and so on.

Similarly, consider another matrix of weights W_K also of dimension (8192 X 8192). This is the ***Key Projection Matrix*.** This transforms those **same** features to represent what context a token provides. $\mathbf{K} = X.\ W_K$.

Thus, the matrices **Q** and **K** are really two different perspectives of the input stream!

And finally, consider a matrix of weights W_V also of dimension (8192 X 8192). This is the ***Value Projection Matrix*.** $\mathbf{V} = X.\ W_V$.

To summarize, these projection matrices: W_Q, W_K, W_V are really transformation matrices. The matrix W_Q, transforms the input features of the tokens into a new space as a matrix **Q** whose features represent what each token in the input stream is seeking. The matrix W_K, projects those same input features into another space as a matrix **K** representing what each token offers to other tokens. The matrix W_V projects the raw input vectors into the final target representation in this evolving semantic space, distilling core meaning that will be carried forward.

Like other matrices, the matrix W_Q (Query) and W_K (Key) and W_V (Value) are initialised by random unconstrained weights.

The table below, summarizes the various matrices used thus far:

Matrix	Rows	Columns	Description
X	32,768	8192	32,768 tokens in a context window each having 8,192 dimensions
W_Q	8192	8192	Weights for queries across each dimension – Query projection matrix. This will transform X into a new 'Queries' space.
W_K	8192	8192	Weights for keys across each dimension – Key projection matrix. This will transform X into a new 'Keys' Space.
W_V	8192	8192	Weights for values across each dimension – Value projection matrix. This will transform X into an evolving semantic space.

Using simple matrix algebra, we can obtain the following additional matrices:

Matrix	Rows	Columns	Description
Q	32,768	8192	$X.W_Q$ This dot product represents what each token seeks from other tokens in the context window
K	32,768	8192	$X.W_K$ This dot product represents what each token provides other tokens across each dimension
V	32,768	8192	$X.W_V$ This dot product is the final token payload in the semantic space
K^T	8192	32,768	K Transpose. Whilst the matrix K is token oriented, K^T is feature oriented! This is derived from K
$Q.K^T$	32,768	32,768	**Attention Matrix.** This is derived by taking the dot product of Q and K^T. This represents how each token influences every other token

Note how the attention matrix is a square matrix whose dimensions equal the number of tokens in our context window. Because of the way it is constructed, tokens appear across both rows and columns, corresponding directly to what each token seeks along one axis and what each token provides along the other axis.

This grid is continuously adjusted to account for the next token. Using the calculus chain-rule these adjustments are passed through to the underlying matrices W_Q and W_K. Because of the bi-directional nature of the attention grid, the matrices W_Q and W_K are forced to assume distinct specialized roles: one as the 'seeker' and the other as the 'provider'! At the conclusion of every pass, the attention grid is re-created from scratch with the freshly revised adjusted weights.

6. The Attention Matrix

Consider the following sentences:

1. *The trophy could not fit into the bag because **it** is too large.*
2. *The trophy could not fit into the bag because **it** is too small.*

To humans it is clear that in sentence 1, the word ***it*** refers to trophy that is too large. Whereas in sentence 2, the word ***it*** refers to the bag that is too small!

The transformer architecture handles this by computing an [n by n] attention matrix where n is the number of tokens in the context window.

LLMs do not process words in isolation but look at the entire context window.

The attention matrix **$Q.K^T$** derived from the dot product of Queries and Keys transposed, represents the relative impact weight of each token on every other token in the sequence.

The **Query (Q)** of the token **'it'** asks the rest of the sequence *what entity am I representing?*

The **Keys (K)** of the tokens 'trophy' and 'bag' respond with their semantic profiles.

In the above example, mathematically, the attention matrix dynamically flips the allegiance of the token ***it*** from the token *trophy* to the token *bag*. The absolute catalyst for the flip is the final word '*large'* vs '*small'*.

In the first sentence culminating with the words 'too large', the query for 'it' is looking for an **object** that would cause an overflow. Whilst in the 'too small' attention head in the second sentence, the query is looking for a **container** lacking capacity.

In the above examples, the semantic embeddings for the token 'bag' would be strongly aligned with being a container whilst the token for trophy would have embeddings that is an object which is **not** a container.

7. Training Cycle – the Forward pass

It is logical to assume that if you speak a language, you understand it.

The main thrust of the training cycle is to build a probabilistic model that can accurately predict the next token given an input stream. Such a model is built by training over terabytes of data. In the process, building a multi-dimensional semantic space (think 8192 dimensions!) where every token is vectorized.

Mathematically, this is achieved by fine-tuning the weights embedded in the matrices W_Q, W_K, W_V and X.

Training is done by passing different input streams of say 32,768 tokens for example. These 32,768 tokens form a 'context window'. It will comprise of multiple sentences of various widths. The last sentence will most likely be truncated. But that does not matter. (*It in fact helps in next token prediction)*.

To work through the math, let us consider a small input stream of just 5 tokens. This is part of the sentence *The cat sat on the* ***mat***. Based on the first five tokens, the weights will be fine-tuned so that the model consistently predicts the token '*mat'*. In this way, by using many different input streams, the model learns to accurately output the next token when fed different sentences.

Step 1: The input matrix (X)

In this example, we have N = 5 tokens: ["The", "cat", "sat", "on", "the"]. Each token is represented by an 8192-dimensional vector. To start with, this vector is initialised randomly.

X = 5 X 8192. (randomly initialised)

Step 2: The Semantic Space Projections (Q, K, V)

We pass the input matrix X (5 X 8192) through three other randomly initialised transformation matrices (W_Q, W_K, W_V) each of dimension 8192 X 8192. The dot product of $\mathbf{X}$ [5 X 8192] with each of these matrices [8192 X 8192] projects the tokens in the semantic space as follows:

Matrix	Description	Formula	Rows	Columns	Size
Q	Queries	$X. W_Q$	5	8192	The dot product has the same number of rows as in **X** and number of columns as in $\mathbf{W}_Q$
K	Keys	$X. W_K$	5	8192	
V	Values	$X. W_V$	5	8192	

Step 3: Computing the Attention Matrix A

Next, we calculate the attention or emphasis each token places on every other token. This is done by multiplying the Queries matrix **Q** with the transposed Keys matrix $\mathbf{K^T}$.

Q = [5 X 8192]

$\mathbf{K^T}$ = [8192 X 5]

A = **Q. K**^{T} = [5 X 5]

Step 4: Scaling and SoftMax normalisation

a) **Scaling***:* The Attention matrix in step 3 was obtained by taking the dot product of two matrices i.e. by multiplying matching components and summing them up.

For example,

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i = q_1 k_1 + q_2 k_2 + \ldots + q_{8192} k_{8192}$$

To start with, the variables were initialised with a mean of 0 and standard deviation of 1. (Xavier/He initialization). Now, with 8192 additions in the dot product, the variance and standard deviation will explode. Given that there are 8192 additions, the accumulated variance would scale linearly to 8192. Since the standard deviation is square root of variance, this needs to be scaled back by $\sqrt{8192} \approx 90.5$.

Dividing the final score by this scaling factor, we pull the bloated standard deviation back to 1. This is the scaled attention matrix $\left(\frac{QK^T}{\sqrt{d_k}}\right)$

b) **SoftMax Function:** The scaled down attention grid will now have random positive and negative numbers. The SoftMax function takes the exponential $\mathbf{e^x}$ of each number x. This math does the following:
   - Exponentiation function makes all numbers are positive
   - It amplifies prominent matches
   - Using **e** has neat mathematical properties during back-propagation (derivative of $\mathbf{e^x}$ is also $\mathbf{e^x}$!)

c) Finally, each of the values e^x is divided by sum e^x for all tokens across each row, so they add up to 1 (100%).

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

At the end of this step, we have converted the raw attention matrix to an Attention Probabilities Matrix **A**

Step 5: Extracting the contextual output Z

Z = **A**.**V** We multiply the adjusted attention matrix **A** from step 4, with the randomly initialised payload **V.**

The matrix **A** [5 X 5] is taken from step 4 and the matrix **V** [5 X 8192] is taken from step 2. This dot product results in a matrix **Z** with dimensions [5 X 8192].

Before this step, each token had its payload in isolation with no consideration to other tokens in the context window.

After this step, each tokens' payload has been blended with the attention matrix to provide a context-aware payload!

Step 6: Projecting to the vocabulary (The Logits)

We have our vocabulary of 256,000 tokens that has a randomly initialised payload. This could be represented as a matrix $\mathbf{W}_{\mathbf{vocab}}$ of dimension [256,000 X 8192].

At the end of step 4, we have a context aware payload. In our example of 5 tokens each of 8192 dimensions, this is a matrix **Z** of dimension [5 X 8192].

The **last row** of this matrix is aware of all preceding tokens. We extract this last row and project it onto the entire vocabulary of 256,000 tokens to predict the next token.

Mathematically, Logits = $\mathbf{Z}_5 . \mathbf{W}^T_{\mathbf{vocab}}$

[1X8192] dot [8192X256,000] = [1X256,000].

The output is a massive array of 256,000 scores(logits) one for each token!

The model selects the token with the greatest probability.

8. The Feed-back loop

The final layer has given a probability score to **every single token in the vocabulary**. With everything being in a random state to start with, it is highly likely that our LLM has made a random bad prediction for the next token!

The loss function calculates a single scalar number on how close the correct tokens percentage was to a 100%. (The cross-entropy loss function).

This correction is passed through to the weight matrices: $\mathbf{W_Q}$, $\mathbf{W_K}$, $\mathbf{W_V}$. (Using the calculus chain rule). The calculus automatically scales these updates based on the difference. As a consequence, the master embedding matrix and all of the other matrices get updated simultaneously. The system neatly adjusts itself as one fluid mathematical continuum.

The back propagation signal travels all the way back to the very front layer of the model.

This enables the context aware payload to be neatly recalculated in one hit. By pulling and twisting both, the weights and the token embeddings simultaneously, the training loop is arranging the pieces of a massive co-dependant jigsaw puzzle. It forces the tokens to organize themselves into some semantic neighbourhood that aids in more accurate token prediction.

Our output matrix $\mathbf{Z = A \times V}$ Equation 1

Now, the matrix $\mathbf{V}$ itself equals $\mathbf{X.\ W_V}$. Equation 2

Replacing the value for $\mathbf{V}$ in the equation 1,

$$\mathbf{Z = A \times (X.\ W_V).}$$

9. The Completed Transformer Training Pass

At the end of this forward/backward training pass, the model has achieved the following:

1. Predicted the next token: Whist the initial prediction may be way off, it gets better with each training cycle.
2. Adjusted the weights of the three projection matrices: $\mathbf{W_Q}$, $\mathbf{W_K}$, $\mathbf{W_V}$. You will recall that these started of with initialised un-constrained weights. With training they have become sharper, disciplined feature routers, learning syntax and structural dependencies across our entire corpus.
3. The context vocabulary has been re-written with revised embeddings. The tokens in the processed input stream now share real-world context.

On the very next pass, when processing a next context window, these freshly amended projection weights and token embeddings will be used for new predictions. Each training pass brings the model a step closer to machine intelligence!



10. Closing Thoughts and more Rabbit-holes

10.1 The Causal decoder: Engineering 'Future blindness'

In any conversation, each written word is only aware of what has already been written in a given interaction. The reader is obviously unaware of what words will be used next in the interaction. In our example 5 X 5 attention grid, when the model evaluates the second token for instance, tokens three, four, five do not yet exist! It thus makes zero computational sense to assign and calculate relationships with tokens that have not as yet been written. The LLM architecture in fact, aggressively chokes these future interactions in the attention matrix by driving them to zero!

-0.25	0	0	0	0
0.96	0.34	0	0	0
0.23	0.54	0.32	0	0
-0.15	0.43	0.44	0.85	0
0.16	0.54	0.79	-0.63	0.86

First Token. Tokens 2,3,4,5 not yet known

Second token: First known. Tokens 3,4,5 not yet known.

Mathematically, this forces the massive 32,768 X 32,768 attention grid into a strict lower triangular matrix as seen above!

The system computes raw attention scores across the matrix and immediately applies a 'causal mask' by overwriting future coordinate slots with negative infinity ($-\infty$).

When these masked values hit the SoftMax gate, their exponential values drop to a hard 0.0%. Since $e^{-\infty}$ = zero the attention weight becomes zero.

This simple geometrical truncation saves computational waste and preserves chronological direction of the human language by forcing the model to predict the next token strictly based on past context! It is this adjusted attention matrix that is used in step 5.

10.2 The Context window and Fragmented Thought

One may be led to believe that a training context window of 32,768 tokens forces the LLM into a rigid operational cage – how will it understand smaller text streams of just a few tokens? In fact, the 32K window is the maximum rather than some sort of minimum. This is achieved by two mechanisms:

a) **Causal masking** that we discussed in 10.1 whereby we force the model to learn strictly from past context as it evolves. When a user enters a short prompt, the model instantly overwrites the remaining future slots to negative infinity, forcing the output percentages to zero.
b) **Sequence Packing:** The large 32,768 input stream is stitched together using smaller sentences, short tweets, code blocks, essays etc. In fact, these shorter segments would typically be absolutely un-related! Perhaps a few sentences of sports, interspersed with a recipe for cakes, back to sports etc. This forces the model to calculate context across variable lengths and segments. It's all in the training!

10.3 Multi-Head Attention – Team of Experts

Thus far in the discussions, we have considered a single large Query Matrix, another single large Key Matrix and a third large Value Matrix calculating a single large Attention Map. Forcing everything through one single mathematical map could well result in an information and computation bottleneck.

Multi-head attention solves this problem by deploying 16 separate parallel assembly lines in place of a single production line. The single large input vector is broken down into 16 smaller vectors.

$$\frac{8,192 \text{ total dimensions}}{16 \text{ heads}} = \mathbf{512} \text{ dimensions per head}$$

One can also appreciate, that a single head would 'average out' the learnings!

Conceptually, sixteen different heads each with their own unconstrained weights, allows them to 'learn' their own linguistic and relational nuances independently and simultaneously.

Each individual Head would yield its own matrix of size 32,768 X 512.

A simple matrix concatenation operation will combine this back to a single holistic matrix with information fusion.

$$\mathbf{X}_{\text{Concatenated}} = [\mathbf{H}_1 \quad \mathbf{H}_2 \quad \mathbf{H}_3 \quad \ldots \quad \mathbf{H}_{16}]$$

10.4 The Mathematics of 'Understanding'

What we have seen thus far is, a model optimized to predict the next vector in a large 8192 geometrical space. How then does this lead to 'understanding'?

A human understands a dog by its various features: *it is a mammal, it has fur, it barks, it chases balls etc.* In an LLM context, the token 'dog' has a vector representation in this large dimensional vector space that aligns with the concepts of *mammal, fur, barking etc!*

It 'understands' by representing the tokens as a vector in a high dimensional geometric space. A space with sufficiently large number of dimensions to enable the **representation of all concepts** in the universe as a comprehensive hyper-dense framework of a multi-dimensional web of features.

10.5 The Mathematics of 'Conversation'

Given that our model has been fine-tuned and optimized to predict a single token, how can we expect it to carry-on a sustained human interaction?

To bridge this massive chasm between next token vector prediction and actual conversation, AI labs are required to adopt a completely separate secondary and tertiary training architecture – *Supervised Fine-Tuning* and *Reinforcement Learning*.

Step 1: Supervised Fine Tuning (SFT) - Teaching the model the dialogue format

A pre-trained base model is subjected to a post-training pipeline using an orchestrated training template comprising of curated conversational pairs. The context is populated a a rigid template:

[System]: You are a helpful AI assistant

[User]: Can you help write a Python code snippet for loading an XML file

[Assistant]: Most certainly. Here is the code-<*code snippet*> Let me know if you need any more clarification

In this way, the base model learns that when the token [User] appears, that portion of the context forms a query requirement that needs an answer. When the token [Assistant] fires, the model must transition its Value payload into a helpful solution.

SFT teaches the model to be a conversational agent. Whilst the underlying math is similar to next token prediction, because the model has been trained on highly disciplined dialogue templates, the unconstrained weights in the $\mathbf{W_Q}, \mathbf{W_K}, \mathbf{W_V}$ projection matrices learn a new structural rule.

There is obviously a lot of behind-the-scenes tooling for creating such a training harness and an entire training framework. (I understand that players like ***Hugging Face*** are in this space)

Step 2: Reinforcement Learning

Base LLM looks at predicting the absolute next token that minimizes cross-entropy loss. In that sense it is myopic and has no idea where the conversation is headed!

To overcome this myopic vision, modern LLMs undergo **Reinforcement Learning.** The input training stream comprises a pre-orchestrated training data to reshape a base model.

Obviously, a short paragraph does not do justice.

10.6 Multi-modal Convergence: Mapping Images and Sound

We have examined LLM's using text. But how about representing images and sound in the **same semantic space**? How can the picture of a dog or the sound of barking have a vector similar to the token 'dog'.?

Imagine three separate entry-gate pipelines:

1. A text-tokenizer that embeds tokens with an 8192-dimensionional vector
2. A Vision Transformer (ViT): that chops an image into 16X16 pixel blocks and projects those into an 8192-dimensional vector space
3. An Audio transformer that chops a sound spectrogram and projects those tiny waveforms into an 8192-dimensional vector

At step zero, these three pipelines are completely unaligned. The vector of the text token ‘dog’, an image of a golden retriever and the sound of a dog barking are chaotic and pointing to different directions.

To align them, engineers feed modal pairs of matching data, like the image of a golden retriever sitting next to a text caption that says: *A brown dog barking*. The idea is to **maximize the dot product between these pairs of vectors and minimize the dot product between mis-matched pairs.**

The objective is for the training to transform this 8192-dimensional vector space as modality-agnostic. This universal mathematical representation as a vector, does not care if the input stream came from a keyboard, a camera or a microphone!

10.7 Handling Structured Datasets

Every main-stream ERP system boasts AI capabilities. Our discussions thus far, has been based on un-structured data.

However, traditional ERP systems like SAP rely on disciplined data organised as highly structured tables in a database with strict schema and transactional integrity.

In such a realm, our vocabulary comprises of table names and field names and transaction codes in the ERP universe.

Let us assume that they have been allotted a unique integer id.

Thus, for instance, every SAP transaction and every SAP field like MATNR would have a high dimensional vector representation. So too every SAP process!

This entire assembly-line is managed by a pre-trained transformer **SAP-RPT** orchestrated by SAP AI-Core using similar high-dimensional linear algebra across SAP business processes.

Again, a lot more investigation to be done here. Just put it out there for the sake of completeness.

10.7 Multi-lingual Unified Alignment

Traditional Enterprise Systems handle muti-lingual capabilities through painful and complex double-byte double-digit translation tables and text string mappings. The Transformer bypasses this legacy complexity. The massive vocabulary and tokenizers of flagship LLMs natively embed multi-lingual texts within the same semantic space. Thus the English token ‘*Invoice’* and the German ‘*Rechnung’* have similar coordinates, thereby making the architecture language agnostic!

Conclusion

We have just scratched the surface of a fascinating area in Artificial Intelligence. It is these various conceptual de-tours and mathematical rabbit-holes along the way, that make the journey interesting. Whilst I may not have all the answers, it has been fun to write. I do hope the blog has made for interesting reading as well. Happy Learning.